

Exercise 8 Live KPI Feed and Windowed Aggregation

Python + MySQL or SQLite

BEFORE YOU START

Files you need: `kpi_stream_seed.csv` and `maintenance_windows.csv`, both in your Module 9 data folder.

Tools: Python from Miniforge with pandas, and either MySQL Community Server or SQLite (SQLite needs no installation - it is part of Python).

Every output printed in this handout was produced by running this exact code against the shipped data. If your numbers differ, something in your steps differs.

What you are building and why

Everything you have built so far reads a file that has stopped changing. A telecom KPI feed does not stop changing, and the two things that break first when data moves are the ones you will fix here: a loader that cannot be run twice safely, and an aggregate that quietly destroys the peaks you were trying to detect.

By the end of this exercise you will have a database table that grows while you watch it, a window query that gives you the true peak, and a written answer to one question that most people get wrong.

Deliverables

- A populated `kpi_readings` table with a unique constraint that survives a repeated load.
- A tumbling-window query at 15 minutes and at 60 minutes, both runnable while the feed is appending.
- A short written note answering: of the 69 blank latency values, how many are defects, and how do you know?

Step 1 - Create the table with a unique constraint

The unique constraint is the whole point of this step. Without it, every re-run of your loader doubles the row count, and every count-based alert you build later is wrong by a factor nobody notices.

```
import sqlite3, os
import pandas as pd

DATA = "."          # folder holding the Module 9 CSV files
DB = "telecom_rt.db"

con = sqlite3.connect(DB)
con.executescript("""
CREATE TABLE IF NOT EXISTS kpi_readings (
    reading_ts      TEXT    NOT NULL,
    cell_id         TEXT    NOT NULL,
    region          TEXT    NOT NULL,
    latency_ms      REAL,
    throughput_mbps REAL,
    downtime_min    INTEGER NOT NULL DEFAULT 0,
    drop_call_rate_pct REAL,
    UNIQUE (cell_id, reading_ts)
);
CREATE INDEX IF NOT EXISTS ix_kpi_cell_ts ON kpi_readings (cell_id, reading_ts);
```

```
""")
con.commit()
```

ON MYSQL INSTEAD

Replace TEXT with VARCHAR(32) / VARCHAR(20), REAL with DECIMAL(10,2), and use ENGINE=InnoDB. Everything else in this handout is unchanged. The UNIQUE key and the index matter more than the engine.

Step 2 - Profile the file before you load it

Never load a file you have not counted. This takes thirty seconds and saves an hour.

```
raw = pd.read_csv(f"{DATA}/kpi_stream_seed.csv")
print("rows in file      :", len(raw))
print("distinct cells    :", raw.cell_id.nunique())
print("duplicate (cell_id, ts) :", raw.duplicated(subset=["cell_id", "reading_ts"]).sum())
print("blank latency_ms    :", raw.latency_ms.isna().sum())
print("negative throughput_mbps :", (raw.throughput_mbps < 0).sum())
print("rows with downtime > 0 :", (raw.downtime_min > 0).sum())
```

Expected output

```
rows in file      : 4634
distinct cells    : 12
duplicate (cell_id, ts) : 26
blank latency_ms   : 69
negative throughput_mbps : 7
rows with downtime > 0 : 28
```

Step 3 - Classify the blanks before you decide what to do

STOP HERE AND READ THIS BEFORE WRITING ANY CODE

You have 69 blank latency values. The instinct is to treat all 69 the same way - impute them, or drop them, or raise a ticket against the collector.

That instinct is wrong, and this step exists to prove it. Some of those blanks are the correct value.

```
mw = pd.read_csv(f"{DATA}/maintenance_windows.csv")
raw["reading_ts"] = pd.to_datetime(raw.reading_ts)
mw["window_start"] = pd.to_datetime(mw.window_start)
mw["window_end"] = pd.to_datetime(mw.window_end)

in_maint = pd.Series(False, index=raw.index)
for _, w in mw.iterrows():
    in_maint |= ((raw.cell_id == w.cell_id) &
                (raw.reading_ts >= w.window_start) &
                (raw.reading_ts < w.window_end))

blank = raw.latency_ms.isna()
print("blank inside a maintenance window :", int((blank & in_maint).sum()))
print("blank outside any window      :", int((blank & ~in_maint).sum()))
```

Expected output

```
blank inside a maintenance window : 22
blank outside any window          : 47
```

WHAT THIS MEANS

The 22 blanks inside a maintenance window are not missing data. The cell was switched off by an engineer holding a change ticket. There was no traffic, so there was no latency to measure. A NULL is the honest record of that.

The 47 blanks outside any window are genuine collector losses, and those are the ones worth a ticket. Write both numbers down. You will be asked for them.

Step 4 - Repair only what is genuinely broken

```
clean = raw.drop_duplicates(subset=["cell_id","reading_ts"], keep="first").copy()
print("after de-duplication :", len(clean), "rows")

# negative throughput is a counter rollover, not a measurement
neg = clean.throughput_mbps < 0
clean.loc[neg, "throughput_mbps"] = None
print("negative throughput nulled :", int(neg.sum()))
print("blanks left untouched      :", int(clean.latency_ms.isna().sum()))
```

Expected output

```
after de-duplication : 4608 rows
negative throughput nulled : 7
blanks left untouched      : 66
```

WHY 66 AND NOT 69

Three of the 69 blanks were sitting on duplicated rows and left with their duplicates: two were inside a maintenance window and one outside. Re-run the Step 3 classification on the de-duplicated frame and you get 20 correct and 46 collector losses.

Nothing has been imputed. This is exactly the kind of number that shifts under you when you clean in the wrong order - state the grain, state the operation, then count again.

Step 5 - Load, then load again to prove idempotency

```
clean["reading_ts"] = clean.reading_ts.dt.strftime("%Y-%m-%d %H:%M:%S")
cols = ["reading_ts", "cell_id", "region", "latency_ms",
        "throughput_mbps", "downtime_min", "drop_call_rate_pct"]
recs = clean[cols].astype(object).where(pd.notna(clean[cols]), None).values.tolist()

def load(rows):
    cur = con.cursor()
    cur.executemany(
        "INSERT OR IGNORE INTO kpi_readings "
        "(reading_ts, cell_id, region, latency_ms, throughput_mbps, "
        "downtime_min, drop_call_rate_pct) VALUES (?, ?, ?, ?, ?, ?, ?)", rows)
    con.commit()
```

```
load(recs)
print("after first load :", con.execute("SELECT COUNT(*) FROM kpi_readings").fetchone()[0])
load(recs)
print("after second load :", con.execute("SELECT COUNT(*) FROM kpi_readings").fetchone()[0])
```

Expected output

```
after first load : 4608
after second load : 4608
```

THE POINT OF THIS STEP

The two numbers are identical. That is idempotency, the same discipline you met in Module 3, and it is doing real work here - a live feed will re-send readings, and a pipeline that cannot absorb that safely produces silently wrong counts.

On MySQL use INSERT IGNORE, or INSERT ... ON DUPLICATE KEY UPDATE if you want later values to win.

Step 6 - The feed simulator

Now make the data move. This generator appends one reading per cell per tick, so the table grows while your queries and your dashboard read it.

```
import random, time

last = con.execute("SELECT MAX(reading_ts) FROM kpi_readings").fetchone()[0]
cursor_ts = pd.to_datetime(last)
cells = [r[0] for r in con.execute("SELECT DISTINCT cell_id FROM kpi_readings")]
regions = dict(con.execute("SELECT DISTINCT cell_id, region FROM kpi_readings"))
random.seed(9)

for tick in range(4):      # change to: while True
    cursor_ts = cursor_ts + pd.Timedelta(minutes=15)
    batch = [(cursor_ts.strftime('%Y-%m-%d %H:%M:%S'), c, regions[c],
               round(random.uniform(26, 74), 2), round(random.uniform(90, 300), 2),
               0, round(random.uniform(0.2, 1.4), 3)) for c in cells]
    load(batch)
    total = con.execute("SELECT COUNT(*) FROM kpi_readings").fetchone()[0]
    print(f'tick {tick+1}: appended {len(batch)} at {cursor_ts} -> {total} rows')
    time.sleep(3)
```

Expected output (with random.seed(9) and the four-tick loop)

```
tick 1: appended 12 at 2026-08-07 00:00:00 -> 4620 rows
tick 2: appended 12 at 2026-08-07 00:15:00 -> 4632 rows
tick 3: appended 12 at 2026-08-07 00:30:00 -> 4644 rows
tick 4: appended 12 at 2026-08-07 00:45:00 -> 4656 rows
```

LEAVE THIS RUNNING

Switch the loop to while True and leave it in a second terminal for the rest of the session. MoP 9.2 needs a feed that is genuinely moving - a Grafana panel over a static table teaches you nothing about refresh, and nothing about alerting.

Step 7 - Tumbling windows at two grains

Both queries below run against the incident on CELL-DEL-03 on 2026-08-05 - the one you will detect and alert on in MoP 9.2.

15-minute tumbling window

```
SELECT cell_id,
       substr(reading_ts, 1, 13) || ':' ||
       printf('%02d', (CAST(substr(reading_ts, 15, 2) AS INTEGER) / 15) * 15)
       AS window_start,
       ROUND(AVG(latency_ms), 2) AS avg_latency,
       ROUND(MAX(latency_ms), 2) AS peak_latency,
       COUNT(*)                AS readings
FROM kpi_readings
WHERE cell_id = 'CELL-DEL-03' AND reading_ts LIKE '2026-08-05%'
GROUP BY cell_id, window_start
ORDER BY peak_latency DESC
LIMIT 3;
```

Expected output

```
('CELL-DEL-03', '2026-08-05 20:30', 143.77, 143.77, 1)
('CELL-DEL-03', '2026-08-05 20:45', 142.42, 142.42, 1)
('CELL-DEL-03', '2026-08-05 20:15', 131.23, 131.23, 1)
```

60-minute tumbling window

```
SELECT cell_id,
       substr(reading_ts, 1, 13) || ':00' AS window_start,
       ROUND(AVG(latency_ms), 2) AS avg_latency,
       ROUND(MAX(latency_ms), 2) AS peak_latency,
       COUNT(*)                AS readings
FROM kpi_readings
WHERE cell_id = 'CELL-DEL-03' AND reading_ts LIKE '2026-08-05%'
GROUP BY cell_id, window_start
ORDER BY avg_latency DESC
LIMIT 3;
```

Expected output

```
('CELL-DEL-03', '2026-08-05 20:00', 136.77, 143.77, 4)
('CELL-DEL-03', '2026-08-05 19:00', 109.72, 120.95, 4)
('CELL-DEL-03', '2026-08-05 18:00', 78.89, 89.98, 4)
```

NOTE THE TWO COLUMNS SIDE BY SIDE

In the 20:00 hour the average is 136.77 and the peak is 143.77. Keeping MAX alongside AVG in every window query costs nothing and is the single cheapest defence against the failure in the next step.

Step 8 - The number you write down

```

peak = con.execute("""SELECT MAX(latency_ms) FROM kpi_readings
WHERE cell_id='CELL-DEL-03' AND reading_ts LIKE '2026-08-05%""").fetchone()[0]

four_h = con.execute("""
SELECT MAX(a) FROM (
  SELECT AVG(latency_ms) a FROM kpi_readings
  WHERE cell_id='CELL-DEL-03' AND reading_ts LIKE '2026-08-05%'
  GROUP BY CAST(substr(reading_ts,12,2) AS INTEGER)/4)""").fetchone()[0]

n_raw = con.execute("""SELECT COUNT(*) FROM kpi_readings
WHERE cell_id='CELL-DEL-03' AND reading_ts LIKE '2026-08-05%'
AND latency_ms > 100""").fetchone()[0]

print(f"true 15-minute peak : {peak:.2f} ms")
print(f"highest 4-hour mean : {four_h:.2f} ms")
print(f"understated by      : {(peak-four_h)/peak*100:.1f}%")
print(f"breaches of 100 ms on the raw grain : {n_raw}")
print(f"breaches of 100 ms on the aggregate : {int(four_h > 100)}")

```

Expected output

```

true 15-minute peak : 143.77 ms
highest 4-hour mean : 74.95 ms
understated by      : 47.9%
breaches of 100 ms on the raw grain : 7
breaches of 100 ms on the aggregate : 0

```

THIS IS THE FINDING

Seven genuine breaches on the raw grain. Zero on the 4-hour aggregate. Nobody deleted anything - the quiet hours either side of the incident averaged it out of existence.

Alert near the raw grain. Aggregate only for reporting. Every KPI where the peak is the thing that hurts - latency, drop rate, congestion - obeys this rule.

Completion checklist

#	Check	Evidence
1	Loader is idempotent	Two loads, identical row counts (4608 and 4608)
2	Duplicates resolved	26 removed, 4608 rows remain
3	Blanks classified, not imputed	Raw 22 / 47; after de-duplication 20 / 46
4	Feed is appending	Row count rising in a second terminal
5	Both window grains run	15-minute and 60-minute queries return the values above
6	Aggregation finding recorded	47.9% understatement, 7 breaches vs 0